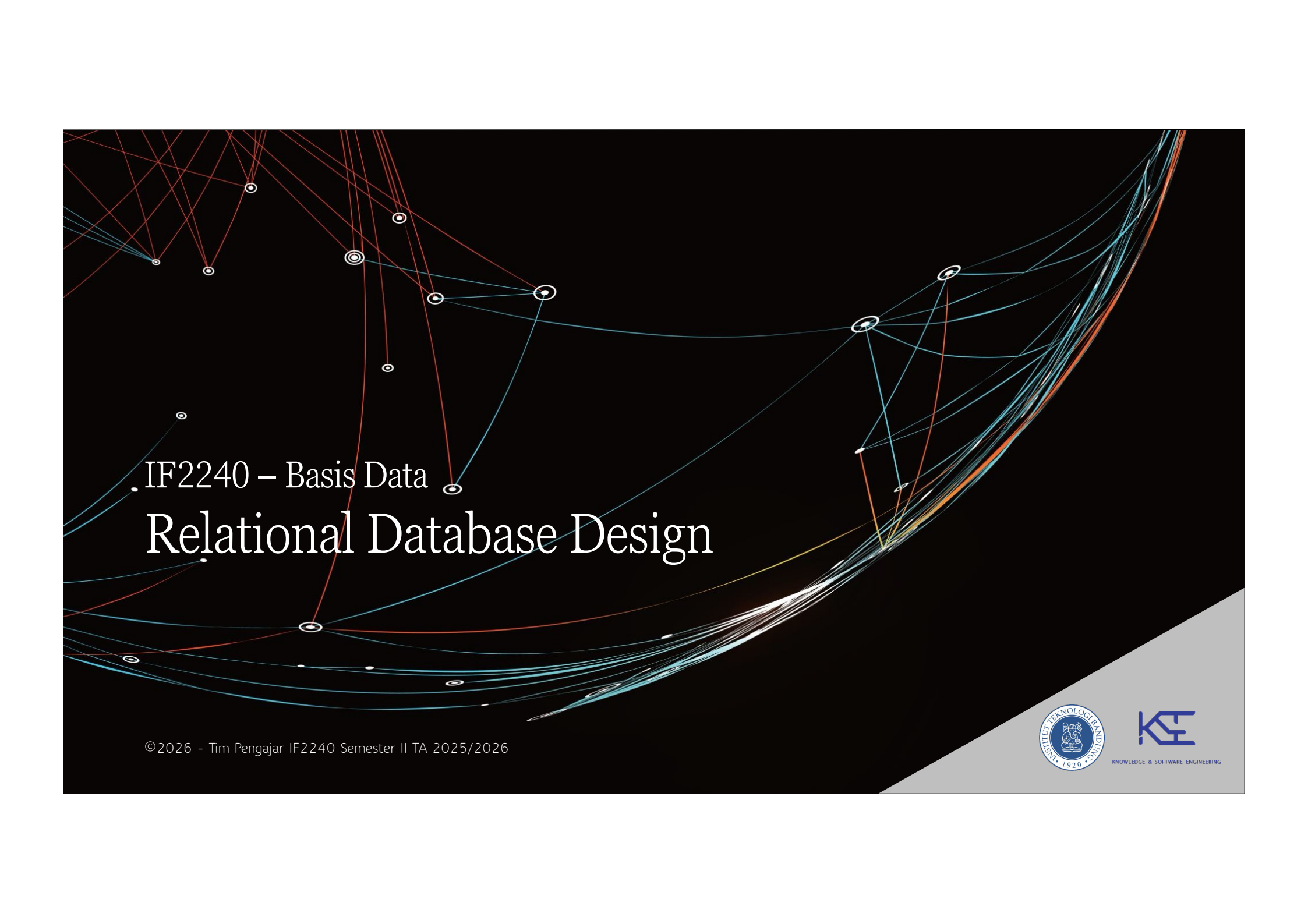


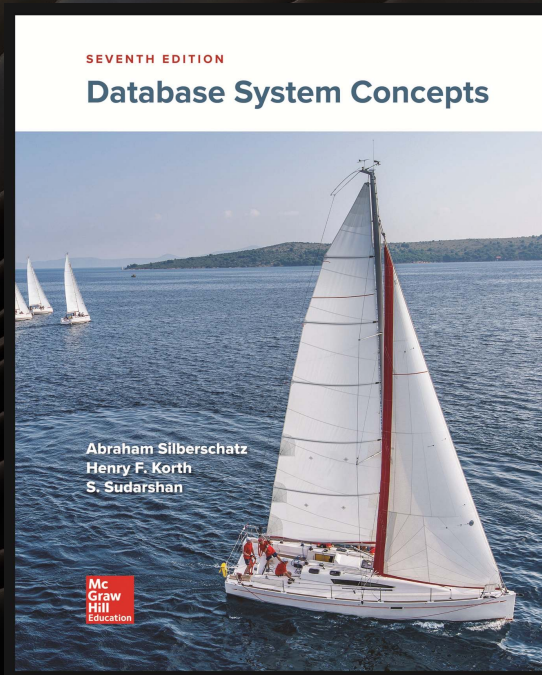


IF2240 – Basis Data Relational Database Design

©2026 - Tim Pengajar IF2240 Semester II TA 2025/2026



KNOWLEDGE & SOFTWARE ENGINEERING



Sumber

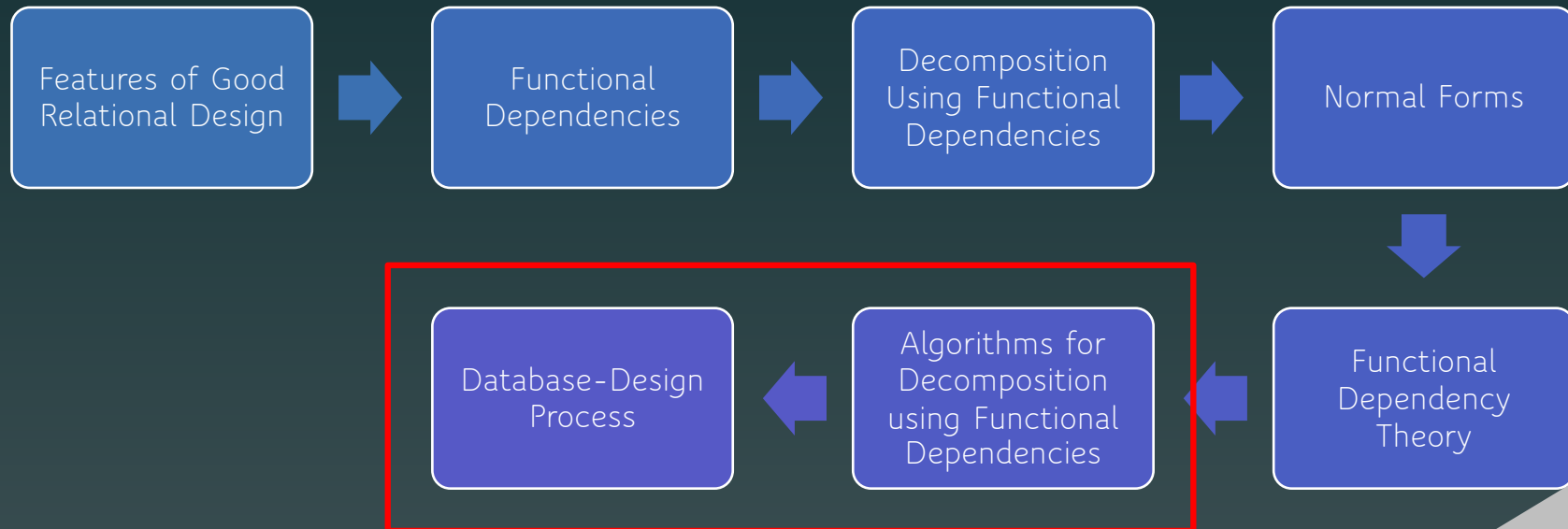
- Silberschatz, Korth, Sudarshan: "Database System Concepts", 7th Edition
 - Chapter 7: Relational Database Design

Capaian

- Mahasiswa dapat menghasilkan desain basis data relasional yang baik berdasarkan prinsip-prinsip yang diberikan



Outline



Algorithm for Decomposition Using Functional Dependencies

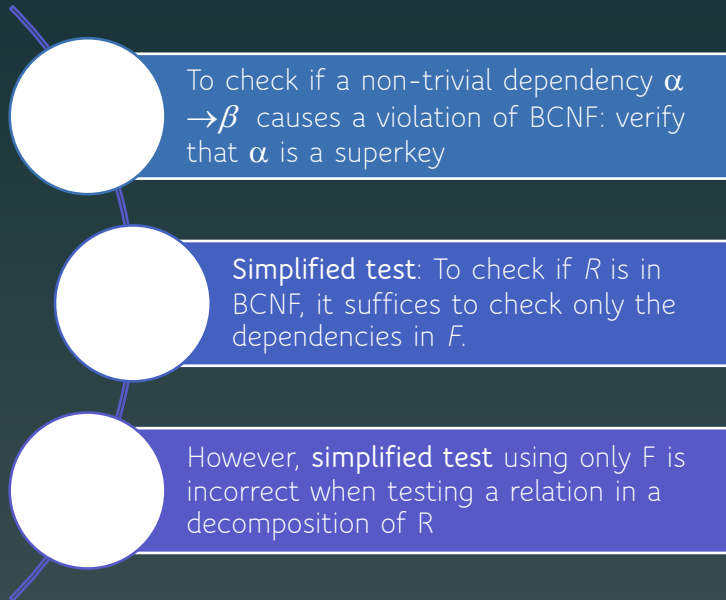
Testing for BCNF

● To check if a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF: verify that α is a superkey

● **Simplified test:** To check if R is in BCNF, it suffices to check only the dependencies in F .

● However, **simplified test** using only F is incorrect when testing a relation in a decomposition of R

Testing for BCNF



- Consider $R = (A, B, C, D, E)$, with $F = \{ A \rightarrow B, BC \rightarrow D \}$
- Decompose R into $R_1 = (A, B)$ and $R_2 = (A, C, D, E)$
- Neither of the dependencies in F contain only attributes from (A, C, D, E) so we might be misled into thinking R_2 satisfies BCNF.
- In fact, dependency $AC \rightarrow D$ in F^+ shows R_2 is not in BCNF.

Testing Decomposition for BCNF

To check if a relation R_i in a decomposition of R is in BCNF

Either

test R_i for BCNF with respect to the **restriction** of F^+ to R_i (that is, all FDs in F^+ that contain only attributes from R_i)

use the original set of dependencies F that hold on R , but with additional test

Testing Decomposition for BCNF

To check if a relation R_i in a decomposition of R is in BCNF

use the original set of dependencies F that hold on R , but with additional test

For every set of attributes $\alpha \subseteq R_i$, check that α^+ (the attribute closure of α) either includes no attribute of $R_i - \alpha$ or includes all attributes of R_i .

If the condition is violated by some $\alpha \rightarrow \beta$ in F^+ , the dependency $\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i$ can be shown to hold on R_i , and R_i violates BCNF.

We use above dependency to decompose R_i

BCNF Decomposition Algorithm

Let F_c be a canonical cover for F
that constraints R .

```
result := {R };  
done := false;  
while (not done) do  
  if (there is a schema  $R_i$  in result that is not in BCNF)  
  then begin  
    let  $\alpha \rightarrow \beta$  be a nontrivial functional dependency that  
    holds on  $R_i$  such that  $\alpha \rightarrow R_i$  is not in  $F^+$ ,  
    and  $\alpha \cap \beta = \emptyset$ ;  
    result := (result -  $R_i$ )  $\cup$  ( $R_i - \beta$ )  $\cup$  ( $\alpha, \beta$ );  
  end  
else done := true;
```

Note: each R_i is in BCNF, and decomposition is lossless-join.

BCNF Decomposition Algorithm

Example

class (*course_id*, *title*, *dept_name*, *credits*,
sec_id, *semester*, *year*, *building*,
room_number, *capacity*, *time_slot_id*)

Functional dependencies:

- *course_id* → *title*, *dept_name*, *credits*
- *building*, *room_number* → *capacity*
- *course_id*, *sec_id*, *semester*, *year* → *building*,
room_number, *time_slot_id*

A candidate key {*course_id*, *sec_id*,
semester, *year*}

BCNF Decomposition:

- *course_id* → *title*, *dept_name*, *credits* holds
- but *course_id* is not a superkey.

- We replace *class* by:

- *course*(*course_id*, *title*, *dept_name*, *credits*) 
- *class-1* (*course_id*, *sec_id*, *semester*, *year*, *building*,
room_number, *capacity*, *time_slot_id*)

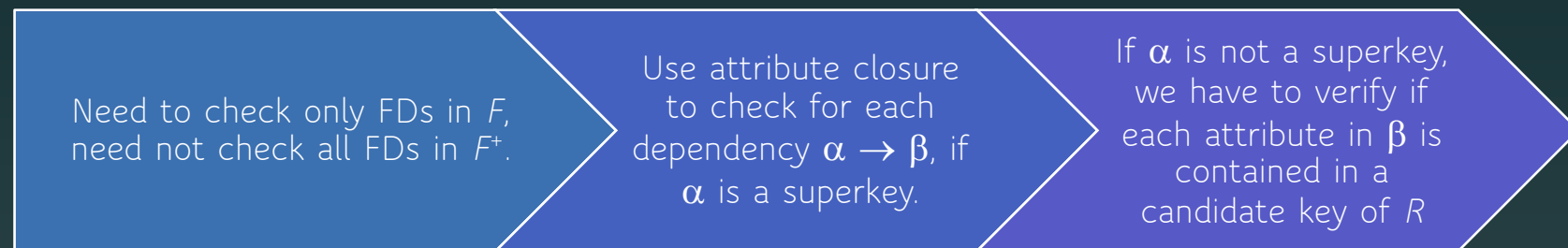
- *building*, *room_number* → *capacity* holds on
class-1

- but {*building*, *room_number*} is not a superkey for
class-1.

- We replace *class-1* by:

- *classroom* (*building*, *room_number*, *capacity*) 
- *section* (*course_id*, *sec_id*, *semester*, *year*,
building, *room_number*, *time_slot_id*) 

Testing for 3NF



This test is rather more expensive, since it involve finding candidate keys

- Testing for 3NF has been shown to be NP-hard

Interestingly, decomposition into third normal form (described shortly) can be done in polynomial time

3NF Decomposition Algorithm

Let F_c be a canonical cover for F
that constraints R .

```
 $i := 0;$ 
for each functional dependency  $\alpha \rightarrow \beta$  in  $F_c$  do
  if none of the schemas  $R_j, 1 \leq j \leq i$  contains  $\alpha \beta$ 
    then begin
       $i := i + 1;$ 
       $R_i := \alpha \beta$ 
    end

  if none of the schemas  $R_j, 1 \leq j \leq i$  contains a candidate key for  $R$ 
    then begin
       $i := i + 1;$ 
       $R_i :=$  any candidate key for  $R;$ 
    end

/* Optionally, remove redundant relations */

repeat
  if any schema  $R_j$  is contained in another schema  $R_k$ 
    then /* delete  $R_j$  */
       $R_j = R_k;$ 
       $i = i - 1;$ 

return  $(R_1, R_2, \dots, R_i)$ 
```

3NF Decomposition Algorithm Example

- Relation schema:

cust_banker_branch = (*customer_id*, *employee_id*, *branch_name*, *type*)

- The functional dependencies for this relation schema are:

1. *customer_id*, *employee_id* $\rightarrow$ *branch_name*, *type*
2. *employee_id* $\rightarrow$ *branch_name*
3. *customer_id*, *branch_name* $\rightarrow$ *employee_id*

- We first compute a canonical cover

- *branch_name* is extraneous in the r.h.s. of the 1st dependency
- No other attribute is extraneous, so we get $F_C =$

customer_id, *employee_id* $\rightarrow$ *type*
employee_id $\rightarrow$ *branch_name*
customer_id, *branch_name* $\rightarrow$ *employee_id*

3NF Decomposition Algorithm Example

- Relation schema:

$\text{cust_banker_branch} = (\text{customer_id}, \text{employee_id}, \text{branch_name}, \text{type})$

- The functional dependencies for this relation schema are:

1. $\text{customer_id}, \text{employee_id} \rightarrow \text{branch_name}, \text{type}$
2. $\text{employee_id} \rightarrow \text{branch_name}$
3. $\text{customer_id}, \text{branch_name} \rightarrow \text{employee_id}$

- We first compute a canonical cover

- branch_name is extraneous in the r.h.s. of the 1st dependency
- No other attribute is extraneous, so we get $F_C =$
 $\text{customer_id}, \text{employee_id} \rightarrow \text{type}$
 $\text{employee_id} \rightarrow \text{branch_name}$
 $\text{customer_id}, \text{branch_name} \rightarrow \text{employee_id}$

- The for loop generates following 3NF schema:

- $(\text{customer_id}, \text{employee_id}, \text{type})$
- $(\text{employee_id}, \text{branch_name})$
- $(\text{customer_id}, \text{branch_name}, \text{employee_id})$

- Observe that $(\text{customer_id}, \text{employee_id}, \text{type})$ contains a candidate key of the original schema, so no further relation schema needs to be added

- Detect and delete schemas, such as $(\text{employee_id}, \text{branch_name})$, which are subsets of other schemas
 - result will not depend on the order in which FDs are considered

- The resultant simplified 3NF schema is:

- $(\text{customer_id}, \text{employee_id}, \text{type})$
- $(\text{customer_id}, \text{branch_name}, \text{employee_id})$

Comparison of BCNF and 3NF

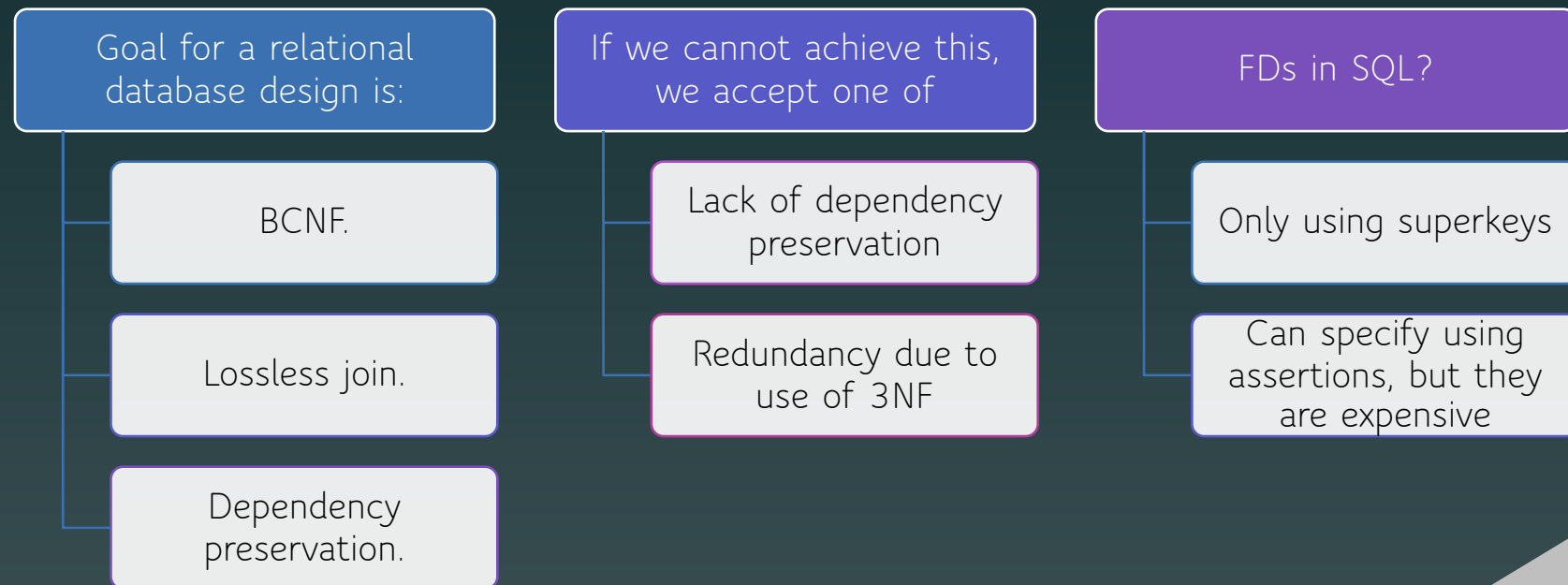
It is always possible to decompose a relation into a set of relations that are in 3NF such that:

- The decomposition is lossless
- The dependencies are preserved

It is always possible to decompose a relation into a set of relations that are in BCNF such that:

- The decomposition is lossless
- It may not be possible to preserve dependencies.

Design Goals



Latihan

Penduduk = (NIK, Nama, TanggalLahir, Alamat, KodeKelurahan, KodeKecamatan, KodeKabKota, KodeProvinsi, KodePos)

- NIK bernilai unik untuk setiap penduduk.
- Setiap kelurahan hanya memiliki sebuah kode pos dan beberapa kelurahan di kecamatan yang sama dapat memiliki kode pos yang sama.

Lakukan normalisasi terhadap Relasi Penduduk hingga menghasilkan relasi-relasi BCNF. Apakah skema yang dihasilkan dependency preserving?



KNOWLEDGE & SOFTWARE ENGINEERING

Overall Database Design Process

Overall Database Design Process

R could have been generated when converting E-R diagram to a set of tables.

R could have been a single relation containing *all* attributes that are of interest (called **universal relation**).

R could have been the result of some ad hoc design of relations, which we then test/convert to normal form.

ER Model and Normalization

When an E-R diagram is carefully designed, the tables generated from the E-R diagram should not need further normalization.



However, there can be functional dependencies from non-key attributes of an entity to other attributes of the entity

Example: an *employee* entity with attributes *department_name* and *building*,

functional dependency *department_name* → *building*



Functional dependencies from non-key attributes of a relationship set possible, but rare --- most relationships are binary

Denormalization for Performance

Example, displaying *prereqs* along with *course_id* and *title* requires join of *course* with *prereq*

Alternative 1: Use denormalized relation containing attributes of *course* as well as *prereq* with all above attributes

faster lookup

extra space and extra execution time for updates

extra coding work for programmer and possibility of error in extra code

Alternative 2: use a materialized view defined a *course* ⋈ *prereq*

Benefits and drawbacks same as above

except no extra coding work for programmer and avoids possible errors

Other Design Issues

Examples of bad database design

Having several relations *earnings_2014*, *earnings_2015*, *earnings_2016*, etc., all with the schema (*company_id*, *earnings*).

- Above are in BCNF, but make querying across years difficult and needs new table each year

Having one relation:
company_year (*company_id*, *earnings_2014*, *earnings_2015*, *earnings_2016*)

- Also in BCNF, but also makes querying across years difficult and requires new attribute each year.
- Is an example of a wide table
- Used in schema

Earnings (*company_id*, *year*, *amount*)

End of Topic



KNOWLEDGE & SOFTWARE ENGINEERING